Manual de uso

AGR — Práctica 2: Virtualización con Vagrant y QEMU-KVM

Pablo Elvira Calatayud

Índice

- 1. Contenido del repositorio
- 2. Parte 1: servidor web con Vagrant
  - 2.1. Instalación (una sola vez)
  - 2.2. Uso
  - 2.3. Comprobaciones
  - 2.4. Problemas frecuentes
- 3. Parte 2: red del laboratorio con KVM
  - 3.1. Instalación (una sola vez)
  - 3.2. Arrancar el laboratorio (tras cada reinicio del anfitrión)
  - 3.3. Órdenes de gestión
  - 3.4. Pruebas
  - 3.5. Medir el consumo (en el anfitrión)
  - 3.6. Problemas frecuentes
- 4. Regenerar la documentación

1. Contenido del repositorio

```
AGR/
├── parte1-vagrant/Vagrantfile     Parte 1: VM con el servidor web (VirtualBox)
├── parte2/                        Parte 2: red del laboratorio 3103 (KVM)
│   ├── plantilla.xml              XML base de Moodle
│   ├── xml/                       XML de cada VM
│   ├── red.sh                     prepara el anfitrión tras cada reinicio
│   ├── molde/                     preparación de la imagen común
│   └── vm/<vm>/                   configuración de dentro de cada VM
└── doc/
    ├── entregables/               PDF finales (+ fuentes en src/)
    ├── development/               enunciado, plano, medidas, informes
    └── build_pdf.sh               regenera los PDF
```

Requisitos del anfitrión: Linux x86-64 Debian/Ubuntu (probado en Ubuntu 24.04), CPU con virtualización (VT-x/AMD-V) activada en la BIOS, ≥ 8 GB de RAM y ≥ 10 GB libres de disco, y conexión a Internet la primera vez.

```
git clone https://github.com/PabloElviraCalatayud/AGR.git
cd AGR
```

> **VirtualBox (parte 1) y KVM (parte 2) no pueden ejecutarse a la vez.** `parte2/red.sh` descarga VirtualBox y carga KVM; el `Vagrantfile` hace lo contrario antes de `vagrant up` (pide la contraseña de `sudo`). Sólo hay que apagar las VMs de una parte antes de usar la otra: `vagrant halt` o `sudo virsh shutdown <vm>`.

2. Parte 1: servidor web con Vagrant

2.1. Instalación (una sola vez)

```
        # VirtualBox 7.2 (repositorio de Oracle)
        sudo apt install -y wget gpg build-essential dkms linux-headers-$(uname -r)
        wget -O- https://www.virtualbox.org/download/oracle_vbox_2016.asc \
          | sudo gpg --dearmor --yes -o /usr/share/keyrings/oracle-virtualbox-2016.gpg
        echo "deb [arch=amd64 signed-by=/usr/share/keyrings/oracle-virtualbox-2016.gpg]
https://download.virtualbox.org/virtualbox/debian $(lsb_release -cs) contrib" \
          | sudo tee /etc/apt/sources.list.d/virtualbox.list
        sudo apt update && sudo apt install -y virtualbox-7.2
        sudo usermod -aG vboxusers $USER          # cerrar sesión y volver a entrar

        # Vagrant (repositorio de HashiCorp)
        wget -O- https://apt.releases.hashicorp.com/gpg \
          | sudo gpg --dearmor --yes -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
        echo "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com
$(lsb_release -cs) main" \
          | sudo tee /etc/apt/sources.list.d/hashicorp.list
        sudo apt update && sudo apt install -y vagrant

        VBoxManage --version     # sin WARNING
        vagrant --version
```

2.2. Uso

Acción	Orden (desde `parte1-vagrant/`)
Crear y arrancar la VM (1ª vez: descarga e instala todo)	`vagrant up`
Entrar (muestra la configuración del servidor)	`vagrant ssh`
Estado	`vagrant status`
Volver a aplicar el aprovisionamiento	`vagrant provision`
Reiniciar	`vagrant reload`
Apagar	`vagrant halt`
Borrar la VM	`vagrant destroy -f`

Opciones (variables de entorno antes de `vagrant up`):

Variable	Efecto	Ejemplo
`VM_MEMORY`	RAM en MB (por defecto 1024)	`VM_MEMORY=512 vagrant up`
`VM_CPUS`	número de CPUs (por defecto 1)	`VM_CPUS=2 vagrant up`
`VM_GUI`	abre la ventana de VirtualBox (login `vagrant`/`vagrant`)	`VM_GUI=true vagrant up`
`BRIDGE_IF`	añade una interfaz pública en puente con esa NIC física	`BRIDGE_IF=enp3s0 vagrant up`

2.3. Comprobaciones

```
        curl http://localhost:8080                          # anfitrión -> VM (port
forwarding)
        curl http://192.168.56.10:3000                      # red privada
        vagrant ssh -c "curl -s localhost:3000"             # dentro de la VM
        curl "http://localhost:8080/record?
id_nodo=n1&temperatura=24.5&humedad=60&co2=300&volatiles=100"
        # Navegador: http://localhost:8080  y  http://localhost:8080/logs
```

Dentro de la VM: `systemctl status ssr-server`, `journalctl -u ssr-server -f`, `sudo ufw status`.

2.4. Problemas frecuentes

Síntoma	Solución
`No usable default provider` / `vboxdrv kernel module is not loaded`	`sudo /sbin/vboxconfig`; si falla, ver filas siguientes
`modprobe vboxdrv failed` con Secure Boot	Desactivar Secure Boot en la BIOS (o registrar la clave MOK) y repetir `sudo /sbin/vboxconfig`
VirtualBox no compila para un kernel nuevo	Instalar la última versión (`virtualbox-7.2`)
`VERR_VMX_IN_VMX_ROOT_MODE` / `VERR_SVM_IN_USE`	Hay VMs de KVM encendidas: apagarlas con `sudo virsh shutdown <vm>`
`A Vagrant environment ... is required`	Ejecutar desde `parte1-vagrant/`
`Vagrant cannot forward the specified ports` (8080 ocupado)	Liberar el puerto 8080 en el anfitrión o cambiar `PUERTO_HOST` en el `Vagrantfile`

3. Parte 2: red del laboratorio con KVM

La red se construye a mano siguiendo `guia_paso_a_paso.pdf` (pasos 0–12). Este apartado resume cómo usarla una vez construida.

3.1. Instalación (una sola vez)

```
        sudo apt install -y qemu-system-x86 qemu-utils libvirt-daemon-system libvirt-
clients bridge-utils
        sudo systemctl enable --now libvirtd
```

Discos en `/var/lib/libvirt/images/lab3103/`: `base.qcow2` (imagen de Moodle), `molde.qcow2` (software común) y uno por VM.

3.2. Arrancar el laboratorio (tras cada reinicio del anfitrión)

```
        cd ~/AGR/parte2
        ./red.sh
        for vm in srv ra rb rc rd pc1 pc2 pc3 pc4; do sudo virsh start $vm; done
        sudo virsh list --all          # las 9 en "running"
```

`red.sh` carga KVM, crea los 9 bridges (`br-srv`, `br-prof`, `br-ab`, `br-bc`, `br-bd`, `br-red1`…`br-red4`), desactiva el filtrado del cortafuegos sobre los bridges y da al anfitrión su IP de PC Profesor (10.200.0.2) con las rutas al laboratorio. La configuración de dentro de las VMs se conserva en sus discos.

3.3. Órdenes de gestión

Acción	Orden
Ver el estado	`sudo virsh list --all`
Entrar en una VM	`sudo virsh console <vm>` (root / agr; salir con `Ctrl+]`)
Ver sus tarjetas y bridges	`sudo virsh domiflist <vm>`
Ver qué hay en cada bridge	`bridge link`
Apagar ordenadamente	`sudo virsh shutdown <vm>`
Apagar de golpe	`sudo virsh destroy <vm>`
Cambiar el XML de una VM	`shutdown` → esperar `shut off` → `undefine` → `define parte2/xml/<vm>.xml` → `start`

VM	IPs	Acceso a la web
SRV	10.100.0.11	sirve en `:3000`
RA	10.100.0.1 · 10.1.0.1 · 10.200.0.1	
RB	10.1.0.2 · 10.2.0.1 · 10.3.0.1	
RC	10.2.0.2 · 10.0.1.1 · 10.0.2.1	
RD	10.3.0.2 · 10.0.3.1 · 10.0.4.1	
PC1–PC4	10.0.N.10 (gw 10.0.N.1)	`curl http://10.100.0.11:3000/`
Anfitrión	10.200.0.2	navegador: `http://10.100.0.11:3000`

3.4. Pruebas

En un router: `vtysh -c 'show ip route'`. En cualquier VM o en el anfitrión, la prueba de conectividad global (18 IPs + web):

```
        ok=0; fail=""; for ip in 10.100.0.11 10.100.0.1 10.200.0.1 10.200.0.2 10.1.0.1
10.1.0.2 10.2.0.1 10.2.0.2 10.3.0.1 10.3.0.2 10.0.1.1 10.0.1.10 10.0.2.1 10.0.2.10
10.0.3.1 10.0.3.10 10.0.4.1 10.0.4.10; do if ping -c1 -W1 $ip >/dev/null 2>&1; then
ok=$((ok+1)); else fail="$fail $ip"; fi; done; echo "$(hostname): $ok/18, web=$(curl -s -o
/dev/null -w '%{http_code}' --max-time 5 http://10.100.0.11:3000/), fallan:${fail:-
ninguna}"
```

Resultado esperado: `18/18, web=200, fallan: ninguna`.

3.5. Medir el consumo (en el anfitrión)

```
        ps -eo rss,args | grep "[g]uest=" | awk '{for(i=1;i<=NF;i++) if($i ~ /^guest=/)
{split($i,a,"[=,]"); printf "%-5s %6.0f MiB\n", a[2], $1/1024; s+=$1}} END {printf "TOTAL
%6.0f MiB\n", s/1024}'
        free -h
        sudo du -h /var/lib/libvirt/images/lab3103/*.qcow2
        watch "ls -al /var/lib/libvirt/images/lab3103/"
```

3.6. Problemas frecuentes

Síntoma	Causa / solución
`virsh start`: *Cannot get interface MTU on 'br-...'*	Los bridges no sobreviven al reinicio: `./red.sh`
`virsh list --all` vacío	Falta `sudo` (sin él se usa `qemu:///session`)
`virsh define`: *domain already exists with uuid*	`undefine` antes de `define`
Un PC: *Destination Host Unreachable* hacia su puerta de enlace	Su router no está encendido o no tiene la IP en esa tarjeta
Un router: *Destination Net Unreachable*	Faltan rutas en el kernel: `systemctl restart frr` (pasa tras `systemctl restart networking`)
Sin conectividad anfitrión ↔ laboratorio	Comprobar `ip -br a show br-prof` y las rutas `via 10.200.0.1`; volver a lanzar `red.sh`

4. Regenerar la documentación

```
        sudo apt install pandoc chromium-browser     # o google-chrome
        doc/build_pdf.sh                             # doc/entregables/src/*.md ->
doc/entregables/*.pdf
```